PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Documento Tecnico

Presentado a: Yoly Alexandra Guevara Muñoz

Por Aprendiz: Miguel Angel Rivera

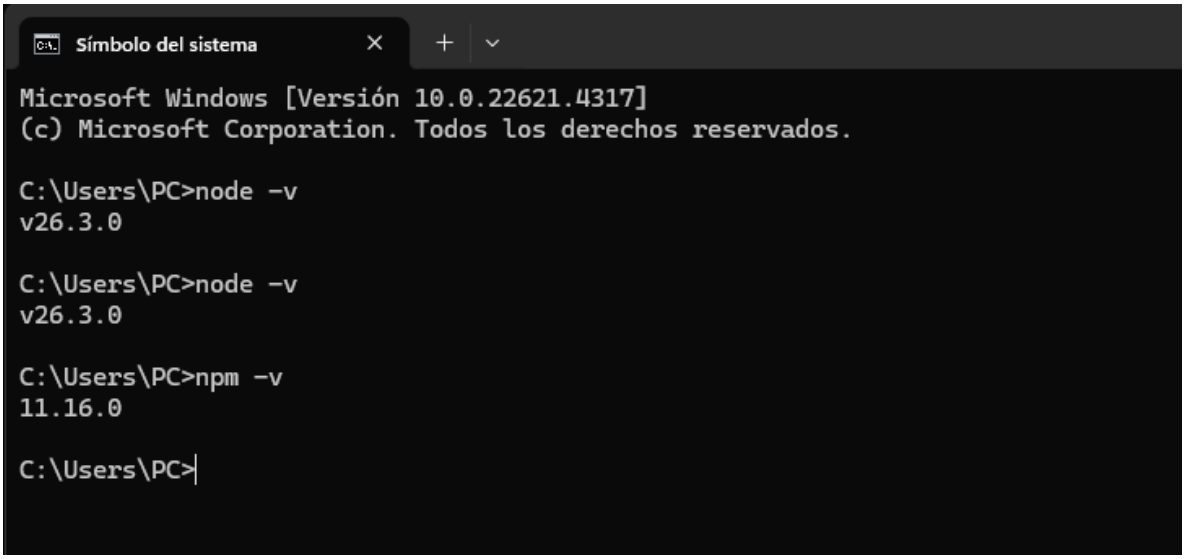
Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

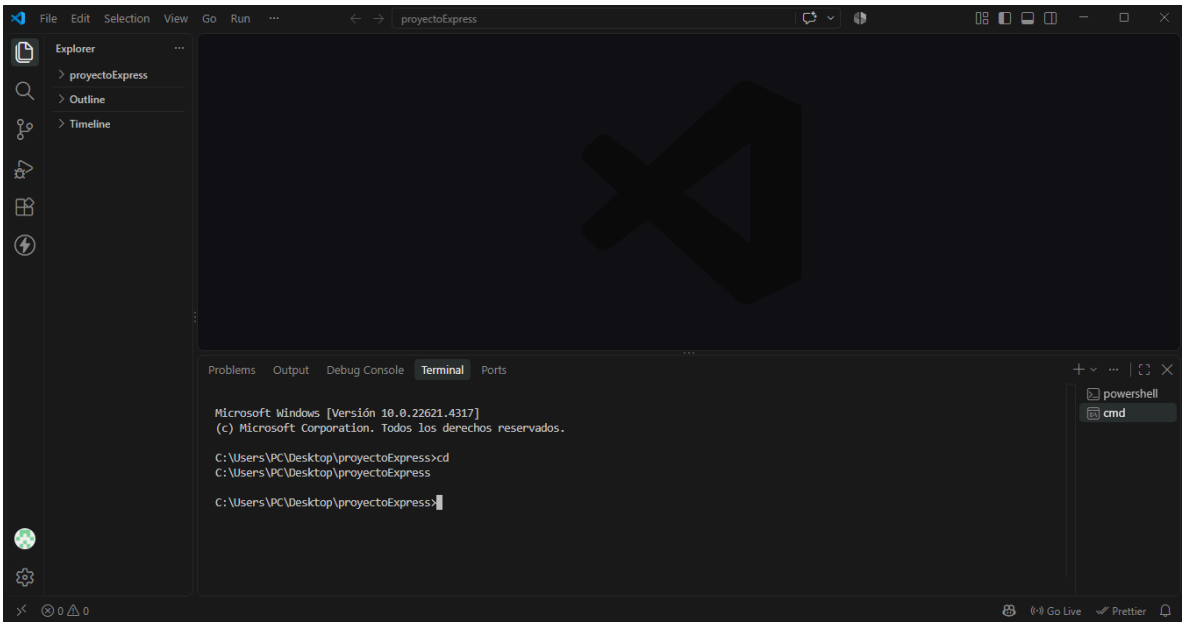
Popayán, día 01 de mes 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

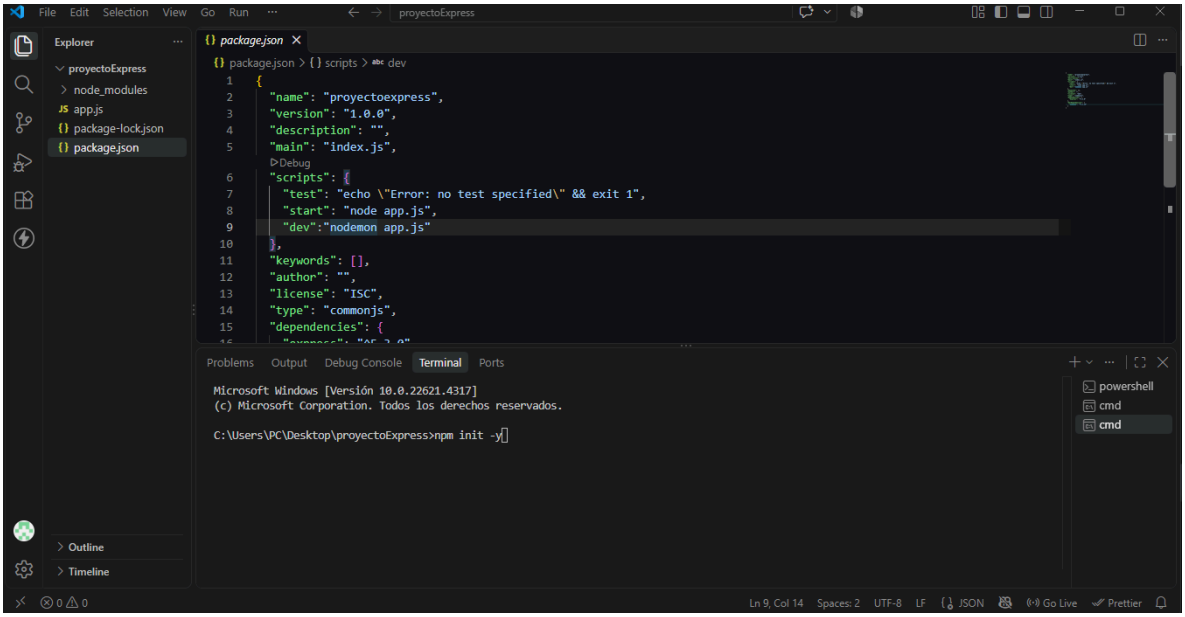
1. **Instalación de Node.js y npm:**
   Node.js permite ejecutar JavaScript fuera del navegador, o sea, en tu computador como servidor. npm es el administrador de paquetes: con él descargas librerías como Express y Nodemon. Si se omite no se podría ejecutar app.js ni usar comandos como npm install o npm start. Tampoco se podría instalar Express, así que no habría forma de crear la API.



2. **Creación de la carpeta y apertura del proyecto:**
   Una carpeta propia mantiene juntos todos los archivos del proyecto (código, dependencias, configuración) y lo separa de otros trabajos. Al abrir el proyecto en el editor te deja ver y editar todo en un solo lugar. si se omite los archivos quedarían regados, sería fácil perderlos o mezclarlos con otros proyectos, y los comandos de npm podrían instalar cosas en una ubicación equivocada.

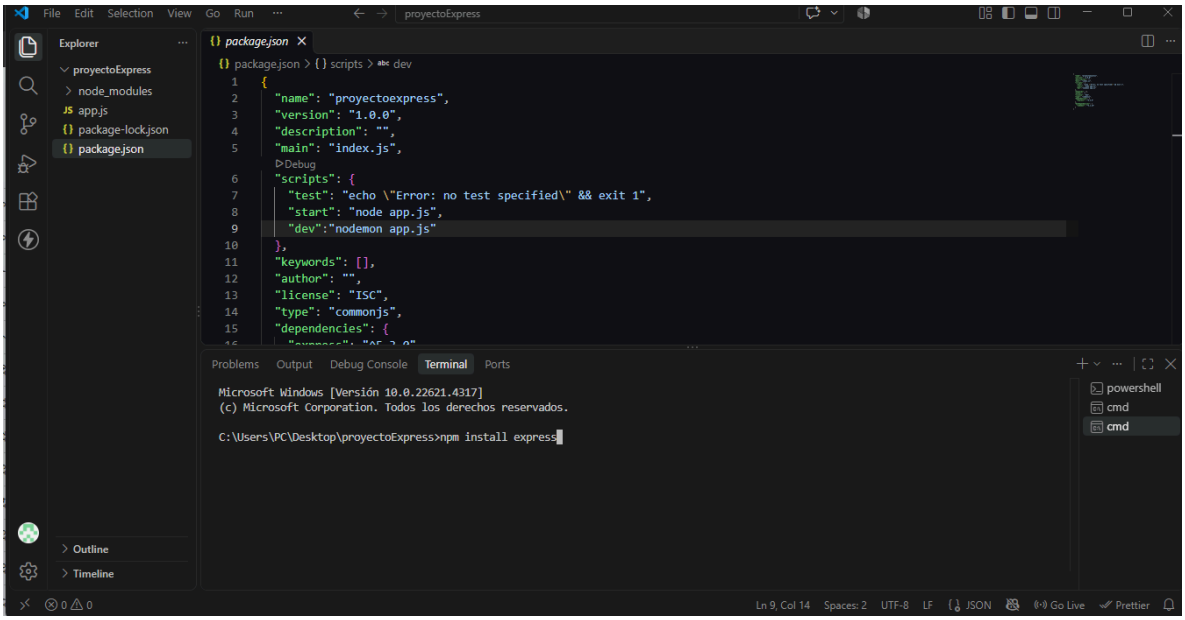


3. **Inicialización del proyecto y package.json:**
   npm init crea el archivo package.json, que guarda el nombre del proyecto, las dependencias instaladas (como Express) y los scripts (npm start, npm run dev).se se omite npm no tendría dónde registrar las librerías ni los scripts. Otra persona que descargue el proyecto no sabría qué instalar, y no podrías usar npm start ni npm run dev.



4. **Instalación de Express:**

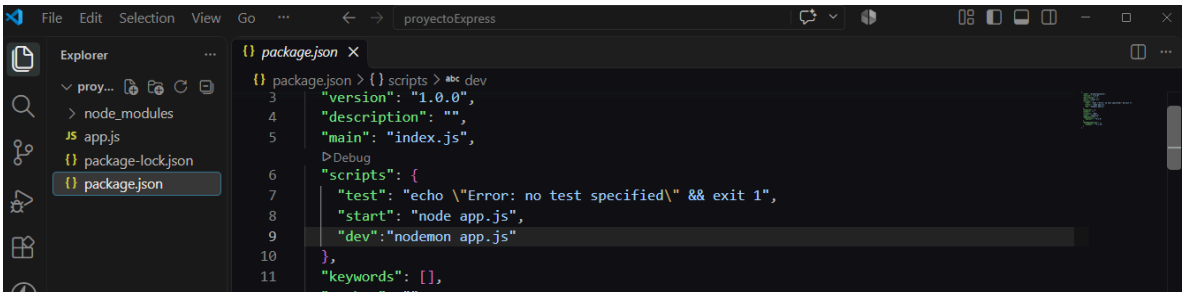
Express es un framework que simplifica crear el servidor, define rutas, maneja middleware y envía respuestas JSON con pocas líneas.



5. **Instalación y configuración de Nodemon:**

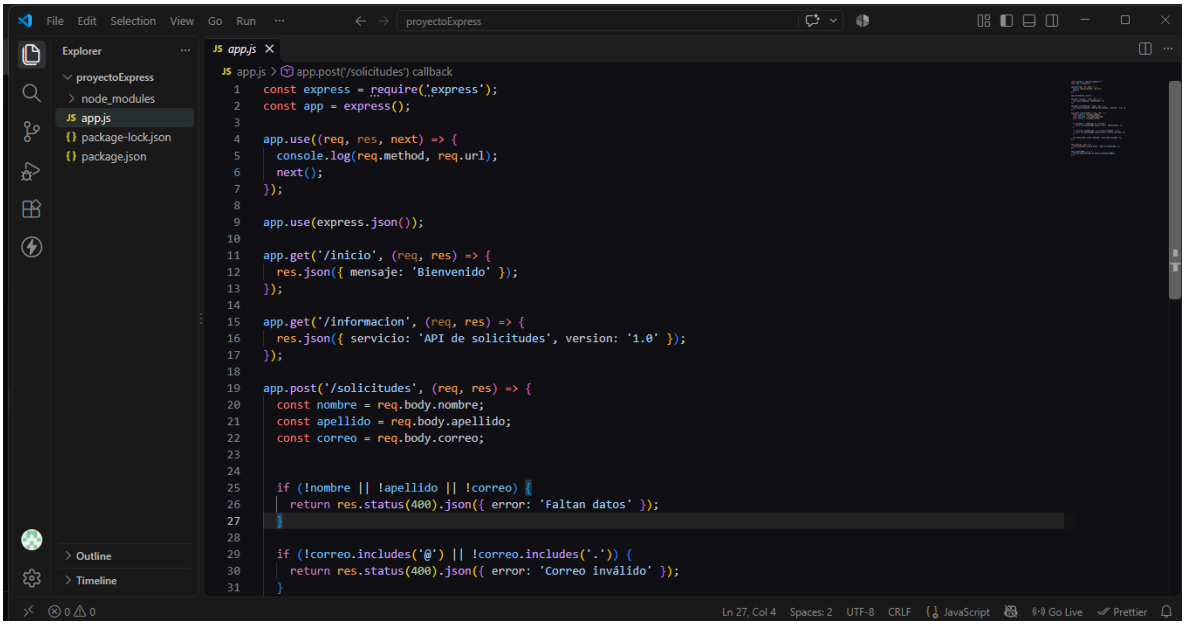
Nodemon reinicia el servidor automáticamente cada vez que guarda un cambio. Se configura con el script "dev": "nodemon app.js" para usarlo con npm run dev. Si se omite cada vez que se modifique el código se tendria que detener el servidor y volver a iniciarlo con npm start. Funciona igual, pero se pierde tiempo y es fácil olvidar reiniciar y creer que tu cambio no sirve.





6. **Archivo principal y puerto:**

   El archivo principal (app.js) es donde empieza la aplicación, ahí se crea el servidor y se definen las rutas. El puerto 3000 es la "puerta" por la que el servidor recibe las peticiones, con app.listen. Si se omite Node no sabría qué ejecutar. Sin listen en un puerto, la app se ejecutaría pero no escucharía a nadie, Postman o el navegador no podrían conectarse y darían error de conexión.



puerto:

```
32
33    res.status(201).json({ mensaje: 'Solicitud recibida' });
34  });
35
36
37  app.use((req, res) => {
38    res.status(404).json({ error: 'Ruta no encontrada' });
39  });
40
41  app.listen(3000, () => {
42    console.log('Servidor en http://localhost:3000');
43  });
```

7. **Middleware para analizar JSON:**
   Cuando el cliente envía datos en JSON, llegan como texto. Este middleware los convierte en un objeto de JavaScript y los deja en req.body, listos para usar en las rutas. Va antes de las rutas porque Express ejecuta todo en orden. Si se omite req.body quedaría vacío y al hacer req.body.nombre el servidor fallaría o no encontraría los datos. La ruta POST siempre diría que faltan campos aunque el cliente los haya enviado.

```
8
9   app.use(express.json());
10
```

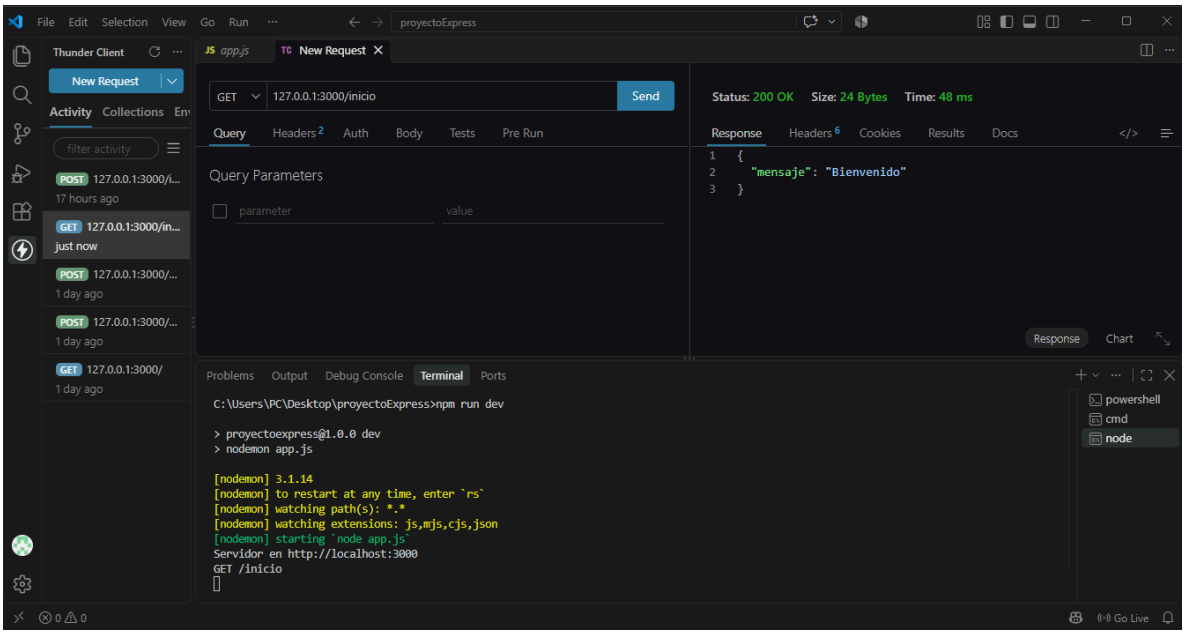
8. **Middleware propio:**
   Sirve para registrar en la consola el método y la ruta de cada petición. Así se puede ver qué está llegando al servidor y comprobar que funciona. Se pone al inicio, antes de las rutas, para que alcance a todas las peticiones, y termina con next() para continuar el flujo. Si se omite no tendría ese registro, y costaría más detectar errores o demostrar qué peticiones llegaron. Si se pone después de las rutas, no registraría las peticiones que ya fueron respondidas. Y si se omite el next(), la petición se queda cargando sin respuesta.

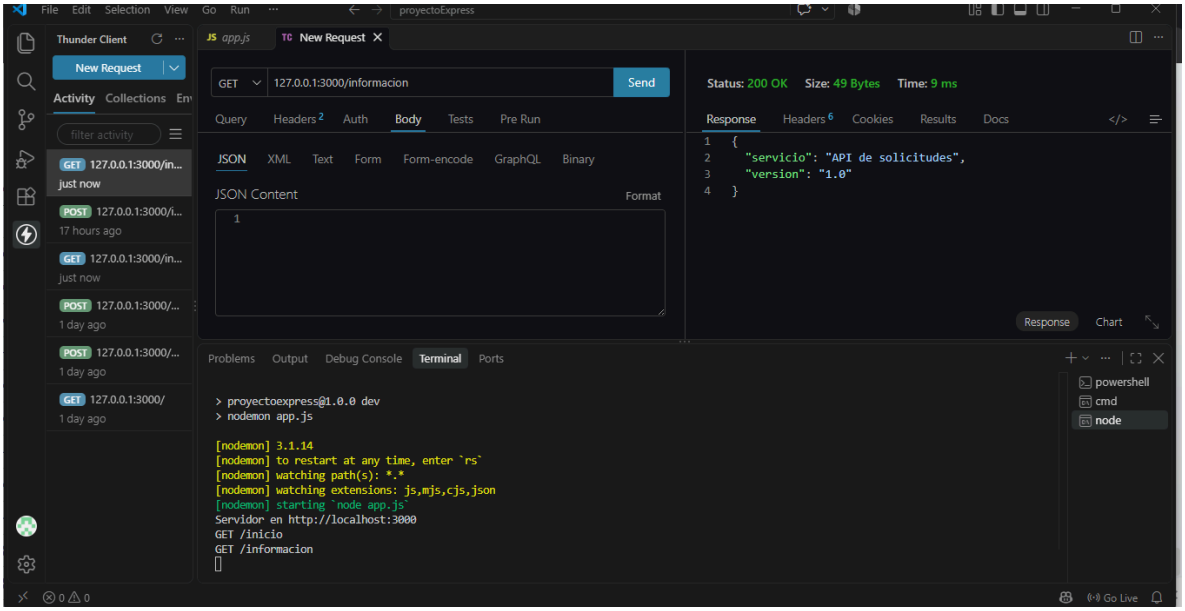
```
3
4   app.use((req, res, next) => {
5     console.log(req.method, req.url);
6     next();
7   });
8
```

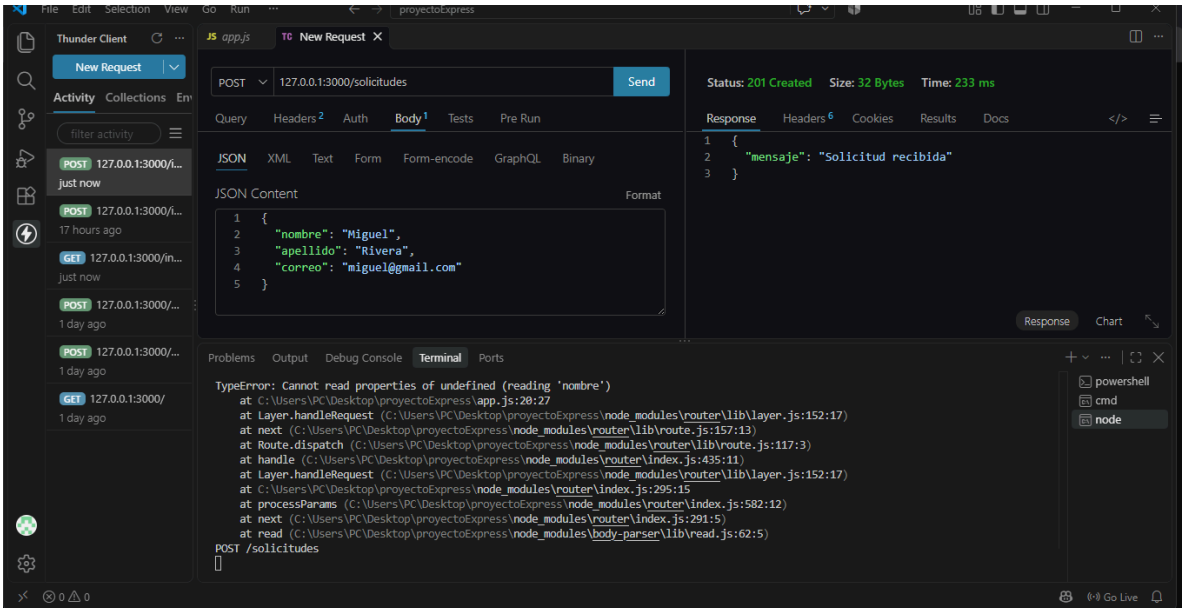
9. **Rutas GET y POST:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

GET sirve para consultar información (/inicio, /informacion), sin enviar datos. POST sirve para enviar datos al servidor (/solicitudes), como el nombre, el apellido y el correo.Si se omite Esa funcionalidad no existiría. Sin las rutas GET no se podría consultar la información del servicio, y sin la POST no se podrían recibir solicitudes. Cualquier petición a esa ruta daría 404.







10. Validación y códigos HTTP:

La validación revisa que los datos estén completos y que el correo tenga un formato razonable. Los códigos HTTP (200, 201, 400, 404) le informan al cliente qué pasó: éxito, creado, error en sus datos o ruta inexistente. Si se omite se aceptarían solicitudes vacías o con datos incorrectos. Y si siempre se responde con éxito, el cliente cree que todo salió bien aunque haya fallado, lo que causa errores difíciles de detectar y datos basura.



```
app.post('/solicitudes', (req, res) => {
  const nombre = req.body.nombre;
  const apellido = req.body.apellido;
  const correo = req.body.correo;


  if (!nombre || !apellido || !correo) {
    return res.status(400).json({ error: 'Faltan datos' });
  }

  if (!correo.includes('@') || !correo.includes('.')) {
    return res.status(400).json({ error: 'Correo inválido' });
  }

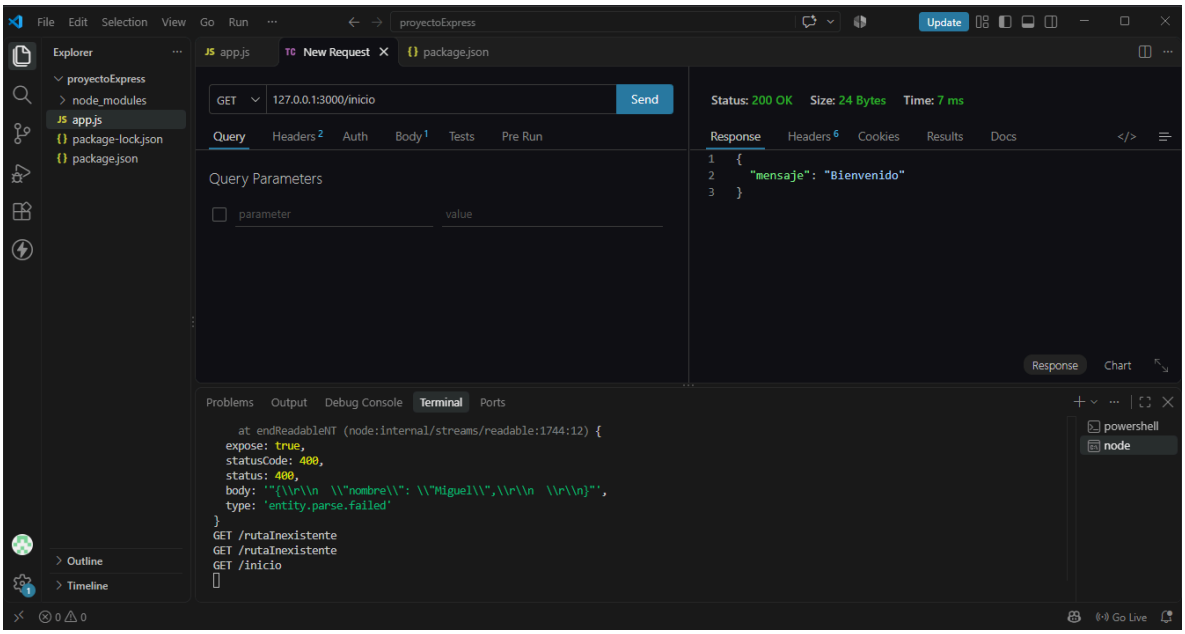
  res.status(201).json({ mensaje: 'Solicitud recibida' });
});
```

11. Pruebas:

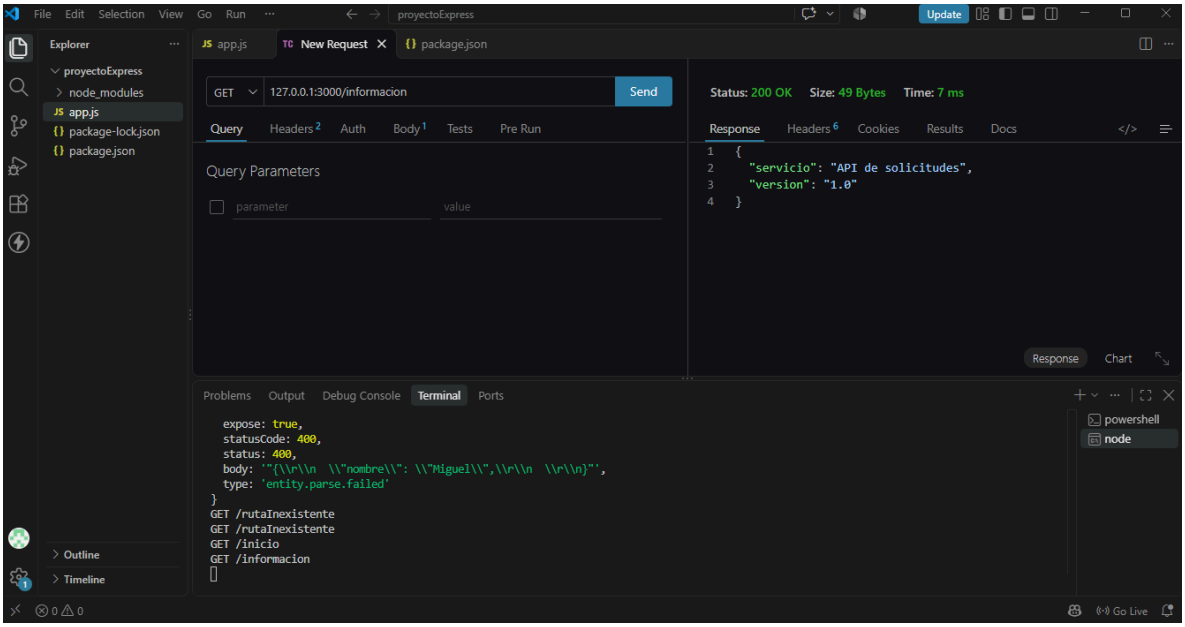
Con Postman se envían peticiones reales a cada ruta y comprueba que responde lo esperado, tanto en casos correctos como incorrectos. Es la evidencia de que la API funciona. Si se omite habrán errores como un body vacío que causa un 500, un

return olvidado o una validación mal escrita pasarían sin ser detectados, y te darías cuenta tarde, por ejemplo al momento de entregar o sustentar.

- **GET /inicio:**

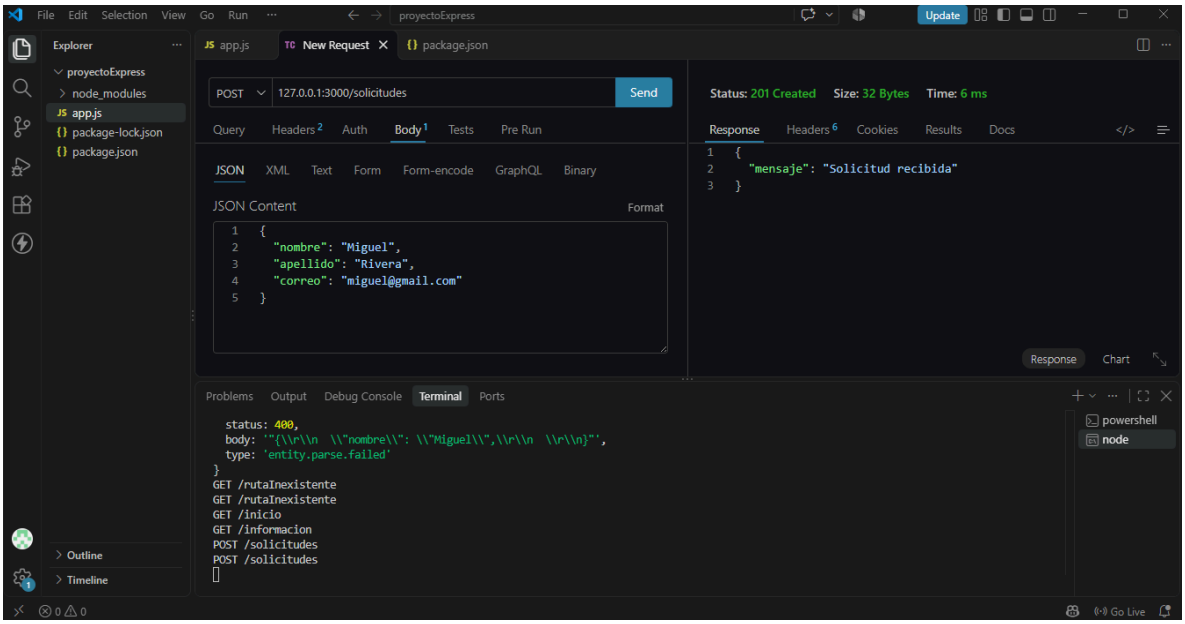


- **GET /informacion:**

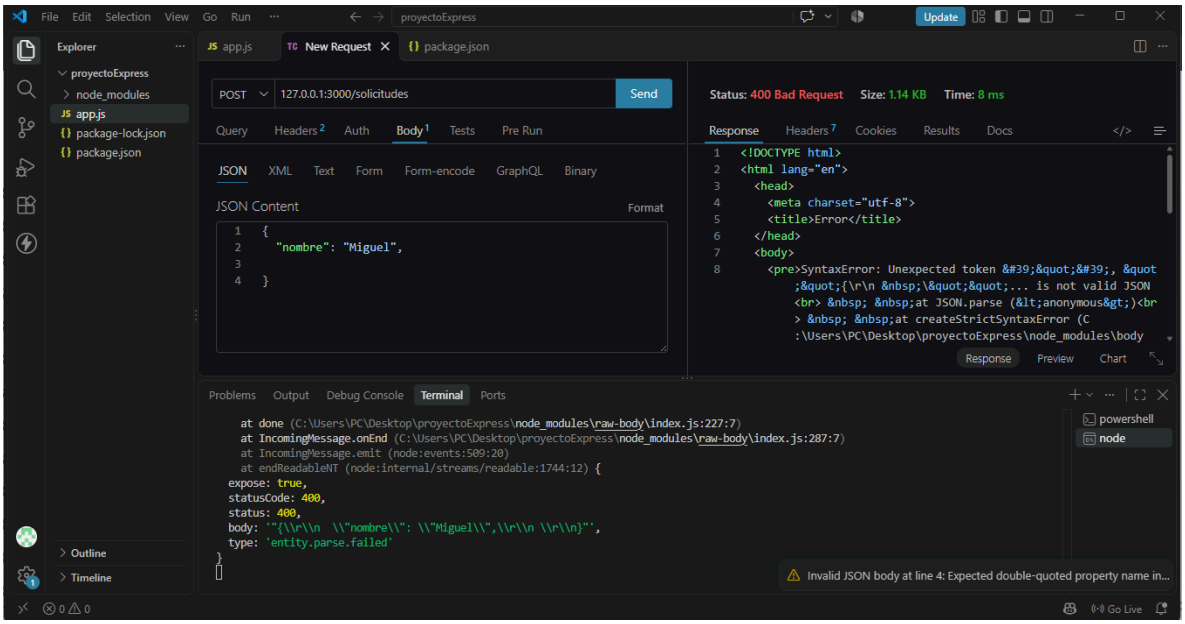


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

- **POST /solicitudes con nombre, apellido y correo válidos:**

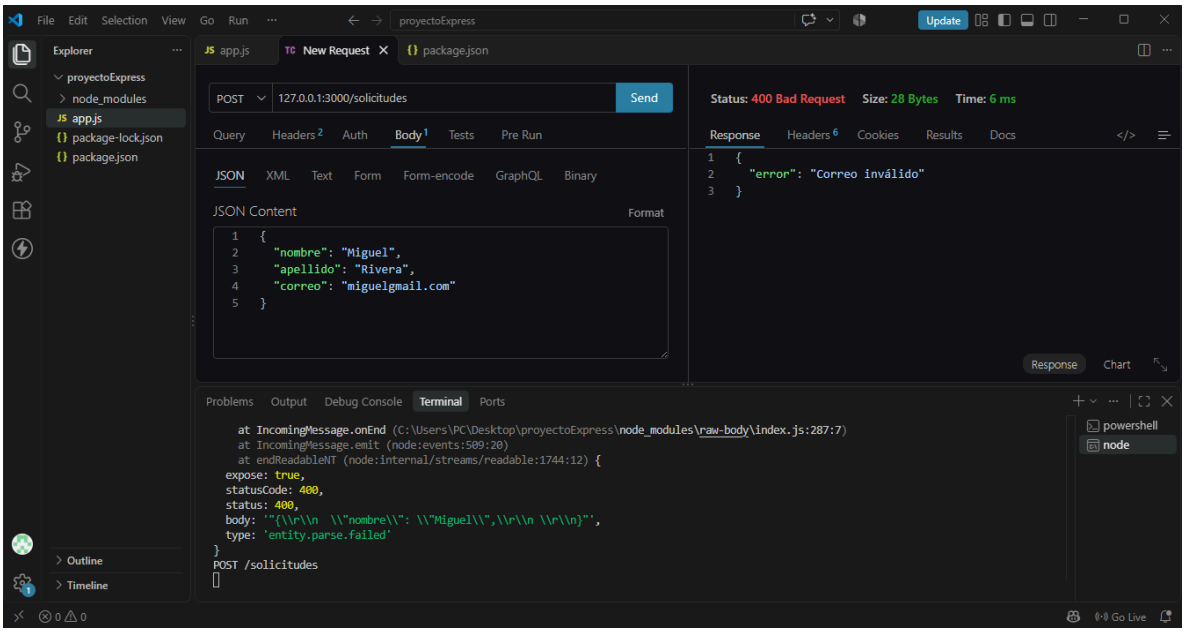


- **POST /solicitudes sin uno o más campos obligatorios:**



- **POST /solicitudes con correo de formato inválido:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



- **Consultar una ruta que no exista y verificar la respuesta 404:**

